

MonPoint MS

Migración de arquitectura monolítica a microservicios

Plan de Trabajo del Equipo

Metodología, cronograma, asignación de responsabilidades y control de avance

Documento	01 de 04 — Plan de Trabajo del Equipo
Proyecto	MonPoint MS — reingeniería arquitectónica de MonPointV2
Equipo	Alejandro (Marquitos) · Victor · Hugo
Periodo	31 de agosto — 6 de noviembre de 2026 (10 semanas) · desarrollo: 28 de septiembre — 6 de noviembre (6 semanas)
Versión	3.4 — cronograma alineado al calendario real de desarrollo
Estado	Cerrado. Solo la secuencia de sprints es adaptativa

Contenido

- 1. Propósito y estado de este documento
- 2. Contexto del proyecto
- 3. Arquitectura objetivo
- 4. Orden de implementación
- 5. Metodología de trabajo
- 6. Roles y responsabilidades
- 7. Cronograma y plan de sprints
- 8. Asignación de trabajo
- 9. Puntos de control
- 10. Criterios de terminado
- 11. Herramientas y convenciones
- 12. Riesgos identificados
- 13. Qué necesita saber cada integrante

1. Propósito y estado de este documento

Este documento define cómo trabaja el equipo durante las diez semanas del proyecto —cuatro de planeación y seis de desarrollo—: qué se construye, en qué orden, quién responde por cada parte, con qué cadencia se sincroniza el equipo y bajo qué criterios se considera que algo está terminado. Cualquier integrante debe poder consultarlo y responder sin preguntar a nadie: qué me toca, para cuándo, y cómo sé que ya acabé.

1.1 Qué está cerrado y qué es adaptativo

Elemento	Estado	Quién puede modificarlo
Alcance del proyecto	Cerrado	Solo el Product Owner, con ajuste de plan
Requisitos y decisiones de diseño	Cerrado	Solo el Product Owner
Historias, criterios de aceptación y estimaciones	Cerrado	Solo el Product Owner
Ownership y asignación	Cerrado	Solo el Product Owner
Catálogo de eventos (desde la semana 4)	Cerrado	Solo el Product Owner
Convenciones, terminado, tablero	Cerrado	Acuerdo explícito de equipo
Secuencia sprint a sprint	Adaptativo	Se ajusta en los puntos de control

Consecuencia práctica

Los criterios de aceptación de las 58 historias están escritos en el Documento 03. La Definición de Listo está satisfecha de origen y el refinamiento desaparece como actividad. Si un criterio resulta ambiguo durante la ejecución, se escala al Product Owner; no se interpreta por cuenta propia.

1.2 Cambios de la versión 3.0

Por requisito de la asignatura, la versión 3.0 incorporó dos cambios estructurales no negociables:

- **MongoDB** sustituye a MariaDB como motor de persistencia de los seis servicios.
- **La comunicación entre servicios es asíncrona**, mediante RabbitMQ. Ningún servicio invoca por HTTP a otro. El cliente sigue comunicándose por HTTP a través del Gateway.

El backlog pasó de 195 a 227 puntos y de 47 a 56 historias.

1.3 Cambios de la versión 3.2

El equipo aprobó sustituir el cliente de escritorio JavaFX por un cliente web en Vue 3: sustitución total, sin cliente dual. El backlog pasó a 234 puntos y 58 historias, con dos historias nuevas: **H-009** (CORS en el Gateway) y **H-087** (empaquetado y despliegue del cliente web).

1.4 Cambios de la versión 3.3

- **Sección 11 ampliada:** se documenta la división de responsabilidad entre Asana y GitHub Projects, exigida por el profesor y acordada por el equipo.
- **Ownership ajustado (sección 8):** ms-auth pasa de Alejandro a Hugo, que ya inició su desarrollo. El empaquetado del cliente web (H-087) pasa de Hugo a Alejandro para equilibrar la carga resultante.

1.5 Cambios de la versión 3.4

La versión 3.3 dejó declarada una divergencia: el cronograma describía el plan original de diez semanas, mientras el desarrollo real arrancó el 28 de septiembre con alcance recortado. Esta versión la resuelve.

- **Sección 7 reescrita por completo:** el cronograma es ahora el calendario vigente, seis sprints del 28 de septiembre al 6 de noviembre, idéntico actividad por actividad al tablero de Asana. Las cuatro semanas previas quedan documentadas como fase de planeación cumplida.
- **Sección 7.4 nueva:** se registra qué quedó fuera al recortar el alcance de 234 a 144 puntos, con la justificación de cada corte.
- **Sección 9:** los tres puntos de control quedan fechados.

Divergencia resuelta

Documento y tablero describen ahora el mismo calendario. La regla de arbitraje de la versión 3.3 —«Asana manda sobre fechas»— deja de ser necesaria y se retira: ambos registros son la misma fuente y deben mantenerse así. Si en un punto de control se ajusta la secuencia, se ajusta en los dos.

1.6 Relación con los demás documentos

Documento	Contenido	Volatilidad
01 — Plan de Trabajo	Cómo trabaja el equipo: metodología, asignación, control	Baja
02 — Requisitos y Alcance	Qué se construye y qué explícitamente no	Nula
03 — Product Backlog	Desglose en historias con criterios de aceptación	Nula
04 — Modelo de Datos	Esquema documental campo por campo	Nula

2. Contexto del proyecto

2.1 De dónde partimos

Dimensión	Estado actual de MonPointV2
Arquitectura	Monolito de capas: Controlador → DAO → Hibernate → MariaDB
Interfaz	JavaFX 21 con vistas FXML y temas CSS
Persistencia	Hibernate ORM 6.4 sobre MariaDB, base de datos única
Comunicación	Llamadas en proceso; no existe red entre componentes
Despliegue	Ejecutable local, instalación manual

2.2 A dónde vamos

Capacidad	Tecnología	Carácter
Servicios	Java 21 · Spring Boot 4.1.1	Obligatorio
Autenticación por token	JWT	Obligatorio
Descubrimiento de servicios	Netflix Eureka	Obligatorio
Punto único de entrada	Spring Cloud Gateway	Derivado
Persistencia	MongoDB (replica set de un nodo) · Spring Data MongoDB	Obligatorio
Comunicación entre servicios	RabbitMQ · Spring AMQP	Obligatorio
Contenerización	Docker y Docker Compose	Obligatorio
Cliente	Vue 3 · Vite · Pinia · vue-router	Sustituye a JavaFX
Servido del cliente	Contenedor nginx sirviendo la compilación de producción	Derivado
Gestión del proyecto	Asana · GitHub Projects	Asana exigido por el profesor
Orquestación de contenedores	Kubernetes	Excluido

2.3 Supuestos de la planeación

- Disponibilidad de 12 horas semanales por integrante.
- Victor y Hugo no tienen experiencia previa en Spring Boot, Docker, MongoDB, mensajería ni arquitecturas distribuidas. Es el factor que más condiciona el cronograma.
- El microservicio de autenticación es obligatorio e invariante.
- Interrupciones previsibles: 16 de septiembre y 2 de noviembre.

Si alguno de estos supuestos deja de ser cierto, debe comunicarse al Product Owner de inmediato.

3. Arquitectura objetivo

Seis microservicios de dominio, cuatro componentes de infraestructura y un cliente web. El cliente habla con el sistema solo a través del Gateway, por HTTP. Los servicios hablan entre sí solo a través del broker, por eventos.

#	Servicio	Responsabilidad	Base lógica	Peso
1	ms-auth	Usuarios, roles, emisión de token	auth_db	Alto
2	ms-inventario	Productos, existencias, reserva de stock	inventario_db	Medio
3	ms-ventas	Ventas, saga de confirmación, folio	ventas_db	Alto
4	ms-clientes	Clientes	clientes_db	Bajo

#	Servicio	Responsabilidad	Base lógica	Peso
5	ms-reportes	Configuración, proyecciones, agregaciones	reportes_db	Medio
6	ms-notificaciones	Registro de eventos y alertas	notificaciones_db	Mínimo

Infraestructura	Función
api-gateway	Punto único de entrada HTTP, enrutamiento y validación local del token
eureka-server	Registro de instancias; el Gateway lo usa para enrutar
rabbitmq	Broker de mensajería: un exchange por dominio, una cola por consumidor, colas de mensajes fallidos
mongodb	Instancia única en replica set; una base lógica y un usuario restringido por servicio
cliente-web	Contenedor nginx que sirve la compilación de producción del cliente Vue

3.1 Decisiones de diseño, resumen operativo

Justificación completa en el Documento 02, sección 5.

Decisión	Qué implica para quien programa
MongoDB en los seis servicios	Spring Data MongoDB, no JPA. La venta y sus líneas son un solo documento.
Replica set de un nodo	Obligatorio para transacciones multidocumento. Sin él, la reserva de stock no funciona.
Decimal128 en valores monetarios	Declararlo explícitamente: por omisión Spring guarda BigDecimal como texto.
Índices declarados	La creación automática está desactivada por omisión; sin declararlos no hay unicidad.
Comunicación por eventos	Ningún servicio llama a otro. Se publica lo que ocurrió; se reacciona a lo que interesa.
Coreografía	No hay orquestador. La saga de venta emerge del intercambio de eventos.
Réplica local en ms-ventas	Ventas guarda su propia copia mínima de productos y clientes para validar sin preguntar.
Proyecciones en ms-reportes	Reportes construye sus datos a partir de eventos, sin llamar a nadie.
Consumidores idempotentes	Todo mensaje puede llegar más de una vez. Procesarlo dos veces no debe tener efecto.
Token validado en el Gateway	El Gateway verifica la firma localmente; no consulta a ms-auth en cada petición.
Venta asíncrona	POST /ventas responde 202 PENDIENTE; el cliente consulta hasta el estado final.
Cliente web en Vue 3	Sustituye a JavaFX. Sustitución total, sin cliente dual.
Token en memoria, no en localStorage	El token vive solo en la tienda de estado. Recargar cierra la sesión; a cambio, no queda expuesto a lectura por script.
CORS en el Gateway	Único componente nuevo por tener el cliente en el navegador.

4. Orden de implementación

Los seis microservicios se diseñan, especifican y documentan desde la primera semana. Lo que se estratifica es el orden de construcción, derivado de dependencias técnicas.

Nivel	Componentes	Criterio
Núcleo	Infraestructura completa (Gateway, Eureka, RabbitMQ, MongoDB), ms-auth, ms-inventario, ms-ventas	El sistema no es entregable sin esto.
Objetivo	ms-clientes, ms-reportes	Planificados y asignados. Se entregan salvo desviación mayor.
Extensión	ms-notificaciones	Especificado desde el inicio. Se implementa con el núcleo estable.

Sin Gateway, Eureka, broker y base de datos operativos, ningún servicio de dominio puede integrarse. Las primeras semanas son ruta crítica pura.

Regla de alcance

Ningún servicio amplía su alcance por iniciativa individual. Tampoco se añaden eventos al catálogo sin aprobación: un evento nuevo es un contrato nuevo entre personas. El riesgo principal de este proyecto no es quedarse corto: es dispersarse.

5. Metodología de trabajo

Scrumban: cadencia semanal fija tomada de Scrum y límite estricto de trabajo en progreso tomado de Kanban. La cadencia es semanal porque, con dos integrantes aprendiendo tecnología nueva, un problema detectado a los catorce días cuesta el veinte por ciento del proyecto; a los siete, la mitad.

5.1 Ceremonias

Ceremonia	Cuándo	Duración	Contenido
Planning	Lunes	30 min	Asignación de historias del sprint. Sin refinamiento: los criterios ya están escritos.
Check-in	Diario	5 min	Estado y bloqueos, por escrito en el canal del equipo.
Review y Retrospectiva	Viernes o sábado	45 min	Demostración de lo terminado y ajuste de la mecánica de trabajo.

5.2 Formato del check-in diario

Ayer: [que **termine**, no que toque]
Hoy: [que voy a **terminar**]
Bloqueo: [que me detiene, o NINGUNO]

Regla de escalamiento

Ningún bloqueo puede durar más de 24 horas sin escalarse. Un integrante atascado tres días en silencio equivale a perder casi una semana-persona. Pedir ayuda pronto es el comportamiento esperado y explícitamente requerido.

5.3 Tablero y límite de trabajo en progreso

LISTO -> EN PROGRESO -> REVISION -> HECHO
(max. 1 por persona) (max. 2)

BLOQUEADO (fuera de flujo, se escala en el check-in)

- **Nadie inicia una historia nueva teniendo una en progreso.**
- Una historia bloqueada se mueve a BLOQUEADO y se escala el mismo día.
- **Si REVISIÓN tiene dos historias, revisar tiene prioridad sobre desarrollar.**
- Una historia llega a HECHO solo si cumple íntegramente los criterios de la sección 10.

6. Roles y responsabilidades

Integrante	Rol	Responsabilidades
Alejandro	Product Owner Tech Lead Facilitador	Propietario del backlog, del alcance y del catálogo de eventos. Resuelve ambigüedades. Decide arquitectura. Desbloquea técnicamente. Interfaz con el profesor. Owner de servicios.
Victor	Developer Owner de servicios	Implementa sus historias según los criterios de aceptación. Revisa lo que le corresponde. Documenta sus servicios.
Hugo	Developer Owner de servicios	Implementa sus historias según los criterios de aceptación. Revisa lo que le corresponde. Documenta sus servicios.

6.1 La figura del revisor

Cada servicio tiene un **owner**, que lo construye y responde por él, y un **revisor**, con tres obligaciones: revisar en menos de 24 horas, ser capaz de ejecutar el servicio localmente y ser capaz de cubrirlo si el owner se ausenta. Ningún servicio debe depender de una sola persona.

7. Cronograma y plan de sprints

El proyecto abarca del 31 de agosto al 6 de noviembre. Las primeras cuatro semanas se dedicaron a análisis, planeación y especificación; el desarrollo de código arrancó el 28 de septiembre. Lo que sigue es el calendario vigente, idéntico actividad por actividad al tablero de Asana.

7.1 Fase de planeación — semanas 1 a 4 (31 ago – 27 sep)

Semanas	Fechas	Resultado
1 – 3	31 ago – 20 sep	Análisis de MonPointV2, definición de alcance y arquitectura objetivo, incorporación de MongoDB y de mensajería asíncrona. Documentos 01, 02, 03 y 04.
4	21 – 27 sep	Sustitución de JavaFX por Vue 3, prototipo del cliente web, diagrama de arquitectura, esqueleto de infraestructura y tablero de Asana.

Desviación respecto al plan original

Ninguna de esas cuatro semanas produjo código ejecutable. El plan original preveía infraestructura operativa y ms-auth completo al cierre de la semana 4. La especificación quedó más sólida de lo previsto, pero la ruta crítica se corrió cuatro semanas y el alcance tuvo que recortarse en consecuencia: de 234 puntos y 58 historias a **144 puntos en 35 actividades**. El recorte está documentado en la sección 7.4.

7.2 Fase de desarrollo — semanas 5 a 10 (28 sep – 6 nov)

Sprint	Fechas	Tema	Actividades	Pts
S1	28 sep – 4 oct	Infraestructura	1.1 MongoDB en replica set · 1.2 Eureka · 1.3 Gateway con enrutamiento · 1.4 Docker Compose del conjunto	16
S2	5 – 11 oct	Seguridad y mensajería	2.1 RabbitMQ con cola de fallidos · 2.2 login con JWT (H) · 2.3 validación de JWT en el Gateway · 2.4 CORS · 2.5 catálogo de eventos congelado · Control 1	19
S3	12 – 18 oct	Servicios de dominio	3.1 modelo de datos (H) · 3.2 productos (V) · 3.3 usuarios y roles (H) · 3.4 clientes (V) · 3.5 eventos de catálogo (V) · 3.6 réplica local en ms-ventas (A)	31
S4	19 – 25 oct	Saga de venta	4.1 registro asíncrono con folio (A) · 4.2 configuración de empresa (H) · 4.3 reserva atómica de existencias (V) · 4.4 resolución de la saga (A) · 4.5 aviso de umbral (V) · 4.6 proyección de ventas (H) · Control 2	31
S5	26 oct – 1 nov	Cliente web	5.1 andamiaje Vue y cliente HTTP · 5.2 consulta de venta y estado (A) · 5.3 inicio de sesión (V) · 5.4 ms-notificaciones (H) · 5.5 pantalla de caja (A) · 5.6 empaquetado nginx (A) · Control 3	31
S6	2 – 6 nov	Integración y entrega	6.1 verificación extremo a extremo · 6.2 README por servicio y endpoints · 6.3 diagramas · 6.4 documento de cierre · 6.5 entrega final	16
Total				144

(A) Alejandro · **(V)** Víctor · **(H)** Hugo. Una actividad sin marca es de trabajo conjunto: los tres integrantes sobre el mismo código.

7.3 Modos de trabajo

Sprints 1 y 2 — Programación en conjunto

Los tres integrantes trabajan sobre el mismo código, rotando cada veinte o treinta minutos. El cuello de botella de estas semanas es conocimiento, no capacidad de producción, y es la única fase en la que nadie puede avanzar por su cuenta sin bloquear a los demás. Al cerrar el sprint 2 existen Gateway, Eureka, broker y base de datos operativos, emisión y validación de JWT funcionando y el catálogo de eventos congelado.

Sprints 3 y 4 — Trabajo paralelo con ownership

Cada integrante implementa las actividades de sus servicios. El catálogo de eventos congelado permite que cada servicio se pruebe publicando eventos simulados, sin esperar a que el servicio productor esté terminado. Es la fase de mayor carga —62 de los 144 puntos— y la que absorbe cualquier retraso acumulado.

Sprints 5 y 6 — Convergencia

El andamiaje del cliente web se construye en conjunto y las pantallas se reparten. El sprint 6 no contiene desarrollo nuevo: es integración, verificación extremo a extremo, documentación y estabilización. Es deliberadamente ligero —16 puntos— porque constituye la reserva del proyecto.

7.4 Qué quedó fuera al recortar el alcance

El recorte de 234 a 144 puntos no eliminó ningún microservicio: los seis se construyen. Lo que se eliminó es profundidad funcional dentro de cada uno.

Qué se eliminó	Por qué
Generación de comprobantes en PDF	Es trabajo de biblioteca, no de arquitectura distribuida. No demuestra ninguno de los requisitos de la asignatura.
Patrón <i>outbox</i> transaccional	Resuelve la pérdida de eventos ante caída del servicio entre el commit y la publicación. Correcto en producción; irrelevante para la demostración y costoso de implementar bien.
Paginación, filtros y ordenamiento en los listados	Volumen de datos de demostración. Se puede añadir después sin tocar la arquitectura.
Gestión de proveedores en ms-clientes	El servicio conserva clientes, que es lo que la saga de venta necesita. Proveedores no participa en ningún flujo entre servicios.
Pantallas de administración del cliente web	Se conservan inicio de sesión y caja, que son las que recorren el sistema completo. El resto se opera por API durante la demostración.

Criterio del recorte

Se eliminó todo lo que no demuestra JWT, Eureka, Docker, MongoDB o comunicación asíncrona. Ninguna de las cinco eliminaciones reduce el número de servicios ni simplifica un flujo entre ellos: el sistema entregado ejercita la arquitectura completa sobre menos funcionalidad. Reducir funcionalidad es recuperable; entregar seis servicios a medio integrar no lo es.

8. Asignación de trabajo

8.1 Ownership de servicios

Servicio	Peso	Owner	Revisor
ms-auth	Alto	Hugo	Victor
ms-inventario	Medio	Victor	Hugo
ms-ventas	Alto	Alejandro	Hugo
ms-clientes	Bajo	Victor	Hugo
ms-reportes	Medio	Hugo	Victor
ms-notificaciones	Mínimo	Hugo	Alejandro
api-gateway e infraestructura	Alto	Equipo completo	—

ms-auth pasó de Alejandro a Hugo en la versión 3.3: Hugo ya había iniciado su desarrollo. Como ms-auth es la plantilla que se replica en los cinco servicios restantes, conviene que lo construya quien ya tiene contexto sobre él.

8.2 Módulos del cliente web

Historia	Módulo	Responsable
H-080	Andamiaje Vue y cliente HTTP con token	Equipo completo
H-081	Inicio de sesión	Victor
H-082	Productos	Victor
H-083	Registro de venta	Alejandro
H-084	Clientes	Victor
H-085	Historial y gráfica	Hugo
H-086	Seguimiento de estado de venta	Hugo
H-087	Empaquetado y despliegue en contenedor	Alejandro

Regla general: cada módulo lo adapta el owner del servicio que consume. Excepciones: H-081 se asigna a Victor por equilibrio de carga, y es viable porque H-080 se construye en conjunto; H-086 se asigna a Hugo porque combina el estado de la venta con el comprobante; H-087 se asigna a Alejandro para compensar la carga que Hugo asumió con ms-auth.

8.3 Carga por integrante

Integrante	Trabajo propio	Pts
Alejandro	ms-ventas, H-083, H-087	33
Victor	ms-inventario, ms-clientes, H-081, H-082, H-084	23
Hugo	ms-auth, ms-reportes, ms-notificaciones, H-085, H-086	34
Compartido	Infraestructura, modelo de datos, verificación, H-080	—

Victor queda diez puntos por debajo. Es deliberado: Alejandro y Hugo llevan las dos piezas de mayor riesgo —la saga de venta y ms-auth con toda la infraestructura de seguridad— y ese margen de Victor es lo que absorbe un atraso si aparece. Si en el primer punto de control resulta que va sobrado, ms-notificaciones es lo más sencillo de reasignarle.

Entregable obligatorio de la fase inicial: el catálogo de eventos

El contrato entre servicios es el catálogo de eventos con el esquema de cada uno. Debe quedar congelado antes de que el trabajo se paralelice. A partir de ahí, productor y consumidor de un evento pueden trabajar en paralelo sin coordinarse, porque ambos programan contra el mismo esquema.

9. Puntos de control

Tres controles con criterios objetivos, al cierre de las semanas de desarrollo 2, 4 y 5. Son el único mecanismo autorizado para modificar la secuencia de sprints, y el ajuste se registra a la vez en este documento y en el tablero de Asana.

Control 1 — Infraestructura operativa · 11 de octubre (crítico)

Criterio: el sistema levanta con una sola instrucción de orquestación; MongoDB responde como replica set; un endpoint protegido responde vía Gateway con token válido y lo rechaza sin él; el cliente web obtiene un token; **un evento publicado por un servicio llega a la cola de otro a través de RabbitMQ**; y el catálogo de eventos está congelado.

Resultado	Acción
Cumplido	Continuar. Se registra la velocidad medida y se recalibra la secuencia.
Parcial (falta orquestación)	Continuar; la orquestación se completa en el sprint siguiente.
Parcial (falta mensajería)	El sprint siguiente se dedica prioritariamente al broker. Nadie inicia historias de consumo de eventos sin él operativo.
No cumplido	ms-notificaciones pasa a <i>especificado, no implementado</i> . El sprint siguiente se reasigna a infraestructura.

Control 2 — La saga de venta funciona · 25 de octubre

Criterio: una venta con existencia suficiente termina CONFIRMADA con el stock descontado, y una con existencia insuficiente termina CANCELADA sin descontar nada. Todo por eventos, sin llamadas HTTP entre servicios.

Resultado	Acción
Cumplido	Continuar según el plan.
No cumplido	Congelar el alcance en cinco servicios y dedicar el sprint siguiente a estabilizar la saga.

Control 3 — Sistema funcionando extremo a extremo · 1 de noviembre

Criterio: desde el cliente web, iniciar sesión, registrar una venta y verla confirmada, sobre el sistema levantado desde cero. Si no se cumple, la última semana se dedica íntegramente a que funcione lo que ya existe.

Principio rector

Ante conflicto entre cumplir el número de microservicios y entregar un sistema que arranca y opera correctamente, gana el sistema que funciona.

10. Criterios de terminado

10.1 Definición de Terminado

Una historia está terminada cuando cumple **todos** los criterios aplicables:

Funcional

- ☐ Los criterios de aceptación de la historia se cumplen sin excepción
- ☐ Validación de entrada en todos los endpoints
- ☐ Manejo de errores con códigos HTTP correctos

Integración HTTP

- ☐ Registrado en Eureka y accesible a través del Gateway
- ☐ Endpoints protegidos y públicos explícitamente diferenciados
- ☐ Verifica el rol en el propio servicio

Mensajería

- ☐ Publica y consume exactamente los eventos que asigna el catálogo, con su esquema
- ☐ Todo consumidor es idempotente: procesar dos veces el mismo evento no tiene efecto
- ☐ Los mensajes que fallan tras los reintentos terminan en la cola de fallidos
- ☐ Ninguna llamada HTTP a otro servicio

Persistencia

- ☐ Acceso exclusivo a su propia base lógica, con su propio usuario
- ☐ Campos monetarios declarados como Decimal128
- ☐ Índices declarados explícitamente
- ☐ Sin credenciales escritas en el código

Contenerización

- ☐ Dockerfile funcional; levanta con la orquestación del conjunto
- ☐ Variables de entorno externalizadas

Calidad y documentación

- ☐ Al menos una prueba automatizada por endpoint y por consumidor principal
- ☐ Revisado y aprobado por el revisor asignado
- ☐ Endpoints y eventos documentados; instrucciones de ejecución local

10.2 Definición de Listo

Todas las historias la satisfacen de origen. Si durante la implementación un criterio resulta ambiguo, se escala al Product Owner en el check-in del mismo día. **No se interpreta por cuenta propia.**

11. Herramientas y convenciones

Función	Herramienta
Gestión del proyecto	Asana — actividades, hitos, controles y entrega
Trabajo técnico diario	GitHub Projects — incidencias y solicitudes de integración
Control de versiones	GitHub , repositorio existente
Consola del broker	Interfaz de administración de RabbitMQ, expuesta solo en entorno local
Inspección de datos	MongoDB Compass o mongosh
Documentación	Markdown en /docs , versionada junto al código
Diagramas	Mermaid, o imagen versionada en /docs/arquitectura

11.1 División de responsabilidad entre Asana y GitHub Projects

El proyecto usa dos herramientas de gestión, no una. Asana es requisito del profesor; GitHub Projects estaba previsto desde la planeación inicial por su vinculación con el código. Se conservan ambas con una división de responsabilidad estricta, porque duplicar información entre tableros es la forma más rápida de que ninguno sea confiable.

ASANA = el plan. Que sigue y cuando.
 GITHUB PROJECTS = el trabajo del día. Que estoy tocando ahora mismo.

	Asana	GitHub Projects
Qué contiene	Las actividades del proyecto agrupadas por sprint, los tres puntos de control y la entrega final. Cada actividad incluye sus criterios de aceptación y las trampas técnicas identificadas.	Incidencias técnicas del día a día: errores encontrados, ramas en curso, solicitudes de integración abiertas.
Granularidad	Actividad: lo que cierra en días. Agrupa una o varias historias del Documento 03.	Incidencia: lo que cierra en horas. Nace del trabajo, no del plan.
Quién lo consulta	El equipo para saber qué sigue. El profesor para evaluar el proceso.	Solo el equipo, mientras programa.
Quién lo modifica	El Product Owner define y ajusta. El equipo marca el avance.	Cualquiera abre y cierra incidencias libremente.
Vínculo con el código	Ninguno. Es deliberado: es una vista de gestión, no de implementación.	Automático. Cada confirmación y cada solicitud de integración queda ligada a su incidencia.

Dónde va cada cosa

Situación	Herramienta
Saber qué me toca esta semana	Asana
Consultar los criterios de aceptación de mi actividad	Asana
Marcar una actividad como completada	Asana
Ver si una actividad está bloqueada por otra	Asana
Reportar un error encontrado mientras programo	GitHub Projects
Abrir o revisar una solicitud de integración	GitHub Projects
Registrar una deuda técnica menor para resolver después	GitHub Projects

Reglas de uso

- **No se duplica.** Una actividad que está en Asana no se copia a GitHub; una incidencia técnica que está en GitHub no se copia a Asana. Dos registros del mismo hecho divergen en cuanto uno se actualiza y el otro no.
- **Asana refleja el plan acordado.** Si alguien considera que una actividad está mal definida o le falta algo, se habla en el check-in; no se edita la descripción por cuenta propia. Esos criterios son lo que evita que cada integrante entienda cosas distintas.
- **Una incidencia de GitHub que revela trabajo no planeado se escala.** Si al resolver un error aparece que falta una actividad completa, eso es un cambio de alcance y lo decide el Product Owner, no se añade al tablero por iniciativa individual.
- **El estado de Asana debe estar al día antes de cada punto de control.** Un tablero que no refleja la realidad no sirve para decidir nada, y los tres controles son decisiones.

Por qué no una sola herramienta

Cargar las 58 historias técnicas en Asana convertiría el tablero en ruido para quien quiere ver el avance del proyecto, y perdería la vinculación automática entre incidencia y código, que es la evidencia más limpia de trazabilidad requisito-implementación. Llevar solo GitHub Projects incumpliría el requisito del profesor y dejaría los hitos y controles sin una vista propia. El costo de mantener dos tableros es bajo mientras la división de responsabilidad se respeta; se vuelve alto en cuanto se empieza a duplicar.

11.2 Topología del repositorio

Monorepo con una carpeta hermana por servicio. El detalle completo, con el árbol de directorios, está en el Documento 04, sección 3.

```
monpoint-ms/
|-- infra/                (eureka-server, api-gateway, docker-compose.yml, mongo-init)
|-- ms-auth/ · ms-inventario/ · ms-ventas/ · ms-clientes/ · ms-reportes/ · ms-notificaciones/
|-- docs/                (eventos, modelo-datos, api, arquitectura)
|-- cliente-web/          (Vue 3, servido por nginx)
`-- legacy/               (codigo de MonPointV2, archivado, no activo)
```

11.3 Convenciones de repositorio

- Una rama por historia, nombrada con su identificador.
- Ningún cambio entra a la rama principal sin aprobación del revisor asignado.
- Las confirmaciones referencian el identificador de la historia.
- Cada servicio vive en su propia carpeta con su Dockerfile, sus instrucciones y una carpeta scripts/ con sus guiones de índices y datos de prueba.
- El catálogo de eventos vive en /docs/eventos y es la única fuente de verdad de los esquemas.
- El esquema de datos vive en /docs/modelo-datos y sigue el Documento 04.
- Los diagramas viven en /docs/arquitectura, con prefijo numérico para conservar el orden.
- El código de MonPointV2 se archiva en /legacy y no se referencia desde ningún servicio nuevo.
- Credenciales por variables de entorno. El directorio de compilación no se versiona.

12. Riesgos identificados

Riesgo	Prob.	Impacto	Mitigación
Infraestructura no lista al cierre de la fase inicial	Alta	Crítico	Control 1 ampliado a broker y base; fase conjunta dedicada a infraestructura.
La saga de venta excede lo estimado	Alta	Alto	Asignada a quien tiene el contexto arquitectónico; Control 2 centrado en ella.
Doble procesamiento de mensajes	Alta	Alto	Idempotencia como criterio de terminado de todo consumidor.

Riesgo	Prob.	Impacto	Mitigación
Replica set no configurado	Media	Alto	Resuelto en el esqueleto de infraestructura; comprobación incluida en el Control 1.
Totales guardados como texto	Alta	Medio	Decimal128 declarado explícitamente y criterio de terminado de persistencia.
Consistencia eventual no comprendida	Media	Medio	Criterios explícitos en las historias del comprobante y del seguimiento de estado.
Curva de aprendizaje de Vue sobre la de Spring, Docker y RabbitMQ	Media	Alto	El equipo no toca Vue hasta terminar la infraestructura; las primeras semanas son solo backend.
Adaptación del cliente subestimada	Alta	Alto	E-08 distribuida a lo largo del proyecto, no concentrada al final.
Los dos tableros divergen	Media	Medio	División de responsabilidad estricta de la sección 11.1; prohibición explícita de duplicar.
Bloqueos no comunicados	Alta	Alto	Check-in diario; escalamiento en 24 horas.

13. Qué necesita saber cada integrante

13.1 Lo que aplica a todos

- **Cadencia:** planning los lunes, review al cierre de la semana, check-in escrito diario.
- **Límite de trabajo:** una sola historia en progreso por persona.
- **Bloqueos:** se comunican dentro de las primeras 24 horas.
- **Revisión:** en menos de 24 horas y con prioridad sobre el desarrollo propio.
- **Herramientas:** Asana para saber qué sigue; GitHub Projects mientras programas. No se duplica información entre las dos.
- **Ningún servicio llama a otro por HTTP.** Si parece necesario, se escala: significa que falta un evento o una réplica local, y eso lo decide el Product Owner.
- **Todo mensaje puede llegar dos veces.** Se programa asumiéndolo.

13.2 Preparación requerida

Temas mínimos, en orden de uso dentro del proyecto.

- Spring Boot: estructura de proyecto, inyección de dependencias, controladores REST.
- Docker y Docker Compose: imagen, contenedor, redes, volúmenes, dependencias entre servicios.
- MongoDB: documentos, colecciones, documentos embebidos frente a referencias, índices, replica set y transacciones.
- Spring Data MongoDB: repositorios, mapeo de documentos, tipo Decimal128.
- Mensajería con RabbitMQ: exchange, cola, routing key, confirmación, reintento, cola de fallidos. Spring AMQP: plantilla de envío y consumidores anotados.
- Conceptos: consistencia eventual, idempotencia, patrón saga.

- JWT: firma y validación.
- Netflix Eureka: qué problema resuelve.
- Vue 3: Composition API, componentes de un solo archivo, reactividad.
- Vite, Pinia y vue-router: proyecto, tienda de estado, rutas.
- Consumo de API desde el navegador: fetch o axios, CORS desde el lado del cliente.

Documento 01, versión 3.4. Reescribe la sección 7 para reflejar el calendario real de desarrollo —seis sprints, del 28 de septiembre al 6 de noviembre— y documenta en la sección 7.4 el recorte de alcance que lo hizo posible. Con esto, documento y tablero de Asana describen la misma secuencia y la regla de arbitraje de la versión 3.3 queda retirada. El resto del contenido no cambia respecto a la versión 3.3. Cerrado salvo la secuencia de sprints, que se ajusta solo en los puntos de control y en ambos registros a la vez.